Data Mining & Informatics Lecture #1

Amin Noroozi
University of Wolverhampton
✉ a.noroozifakhabi@wlv.ac.uk

Module Review

Module Review

- **Description (according to the module specification)**
- This module looks at different data processing and mining techniques and state-of-the-art data processing and mining tools.
- The topics include an introduction to data mining and knowledge discovery process, data collection and processing with various data types from a variety of sources, and social web mining.
- The module will look at a variety of techniques and trends in data mining

Module Review

- **Intended Learning Outcomes (ILO)(according to the module specification)**
- LO1 Utilise all aspects of the knowledge discovery process from data collection and data processing to representation.
- LO2 Apply cutting-edge data mining techniques to extract information from data.
- LO3 Develop your own data mining solution, using a state-of-the-art data mining tool

Module Review

- **Topics**

- Introduction to the module and Python => Week 1
- Data preprocessing => Week 2
- Classification and feature selection => Week 3
- Clustering => Week 4
- Association Rule Mining => Week 5
- Web Scraping and Sentiment Analysis => Week 6

Module Review

- **Module Leaders**

- Amin Noroozi
✉a.noroozifakhabi@wlv.ac.uk

- Anirban Chakraborty
✉a.chakraborty@wlv.ac.uk

- **Timetable**

- Tuesday 12:00-14:00 => Lecture

- Friday 14:00-17:00 => Tutorial

Module Review

- **Assessment**

1. You will be given a number of tasks in each workshop.

2. You will apply data mining methods taught in the lectures and tutorials to the tasks.

3. The submission will be a 3000-word report describing and interpreting the output of your codes and the codes' Python file.

Module Review

- **Assessment**

Important points to consider:

- You should include the screenshots of the output in your report (when applicable)
- It should be clearly shown which code and which description in your report is related to which task (you will lose marks otherwise)
- Run your code before submitting to ensure it works
- Put all files in a ZIP file and submit the ZIP file

Data Mining and Informatics

Data Mining and Informatics

- **What is data mining?**

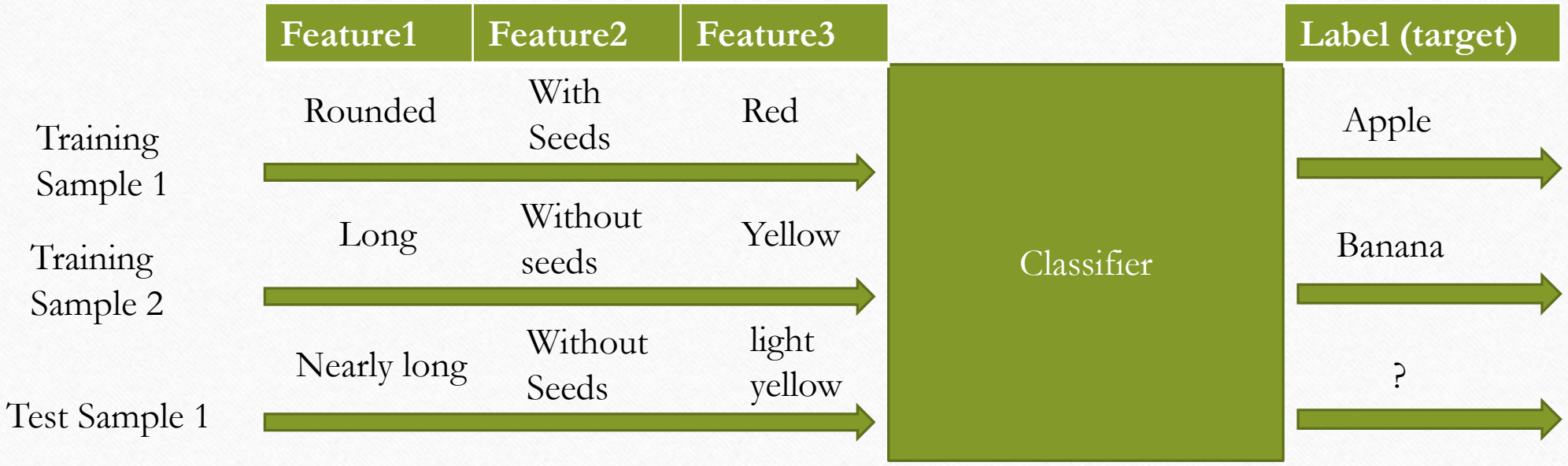
Data Mining and Informatics

- **What is data mining?**

- Data mining is the process of extracting knowledge or insights from large amounts of data using various statistical and computational techniques.
- The data can be structured (table), semi-structured (JSON) or unstructured (webpages), and can be stored in various forms such as databases, data warehouses, and data lakes

Data Mining and Informatics

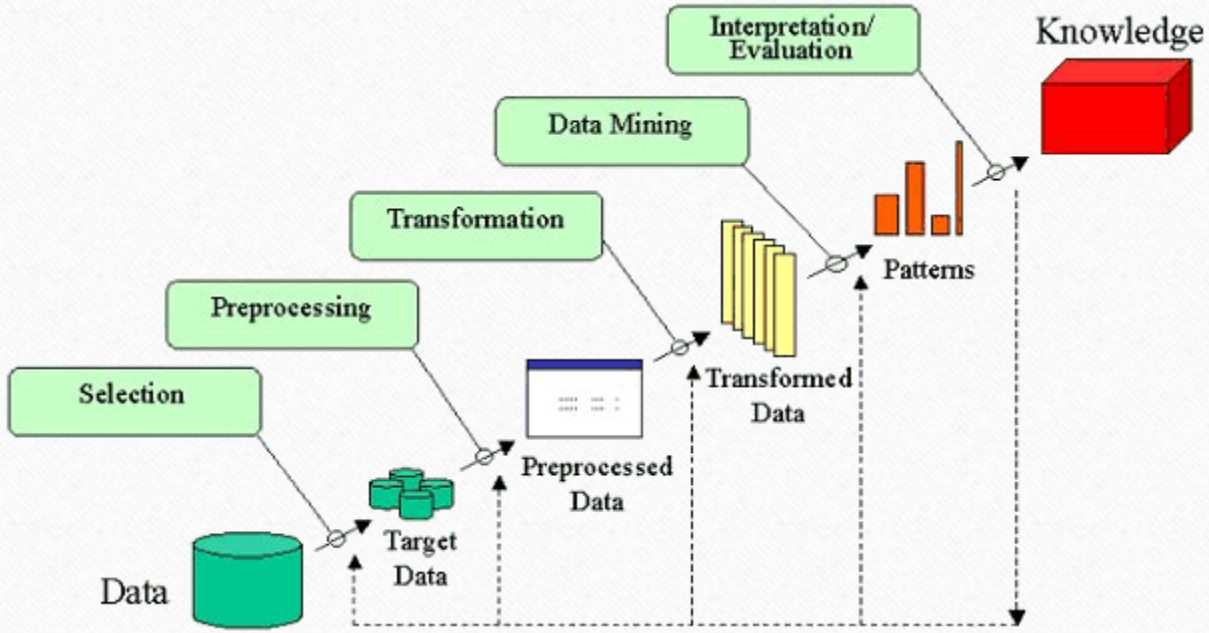
- **Example**



Data Mining and Informatics

- **What is data mining?**

- From another point of view, data mining can be regarded as a step in Knowledge Discovery in Databases (KDD)



Data Mining and Informatics

- **What is data mining?**

- KDD: Knowledge Discovery in Databases, commonly referred to as KDD, is a systematic approach to uncovering patterns, relationships, and actionable insights from vast datasets

- KDD involves multiple steps, from selecting and preprocessing data to the actual process of data mining and finally to the interpretation and use of the results

Data Mining and Informatics

- **What is data mining?**

- *Data selection*: The first step in the KDD process is data selection. This involves gathering data from various sources to form a raw dataset

- *Data preprocessing*: Preprocessing is crucial for ensuring that our dataset is of good quality. This step involves handling inconsistencies, missing values, and noise

- *Data transformation*: Once the data is cleaned, it often needs to be transformed to be suitable for data mining. This might involve scaling, encoding, or feature engineering

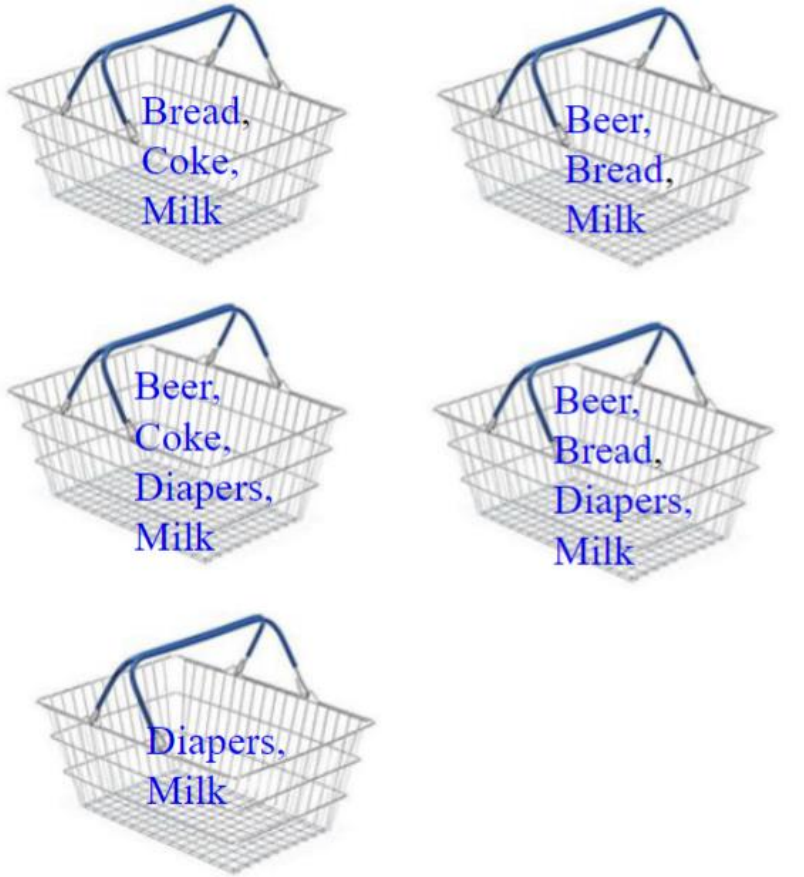
Data Mining and Informatics

- **What is data mining?**

- *Data mining*: The primary goal of data mining is to discover hidden patterns and relationships in the data that can be used to make informed decisions or predictions
- There is no consensus on data mining definition and different sources consider different techniques as data mining.
- But a significant number of sources at least mention the following three techniques:
- **Clustering, classification, and association rule mining**
- Some sources also discuss regression, artificial intelligence and anomaly detection methods.

Data Mining and Informatics

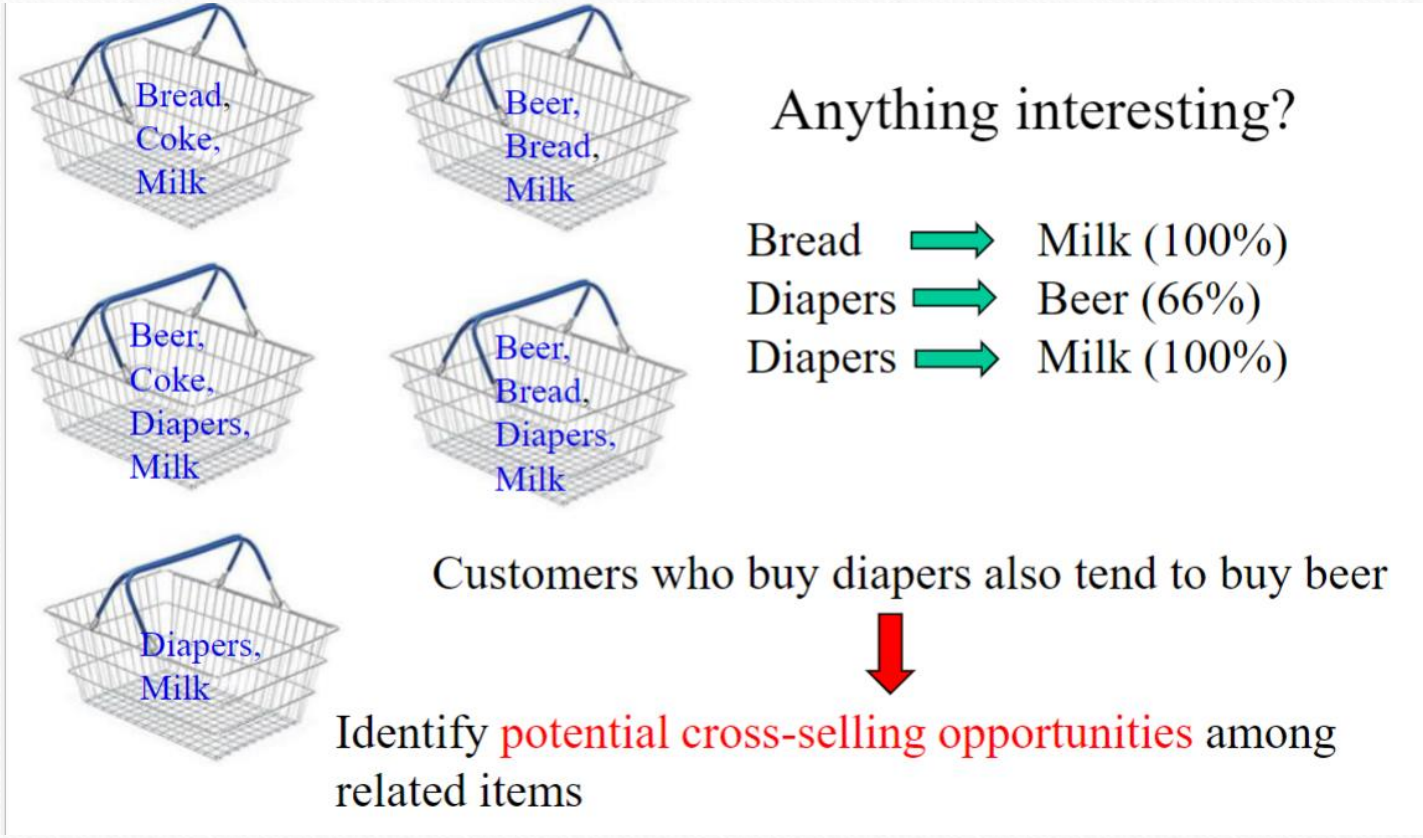
- **Example**



Anything interesting?

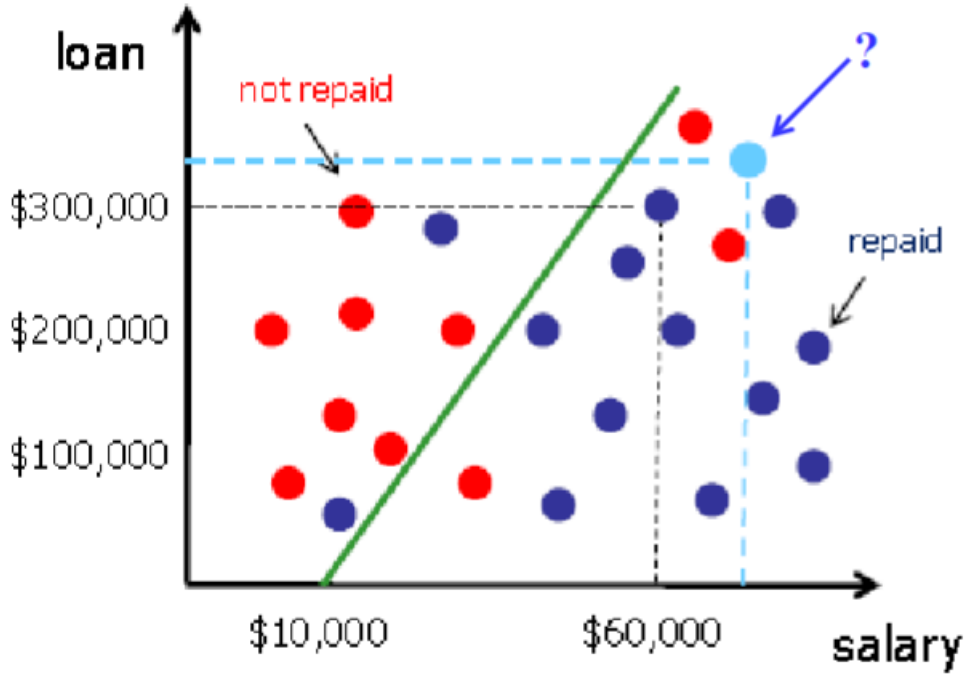
Data Mining and Informatics

- **Example**



Data Mining and Informatics

- **Example**



Given: a loan application

Problem: predict whether the bank should approve the loan?

Data: records from other loans

Data Mining and Informatics

- **Example**

- Twitter Sentiment Analysis
  - Twitter data gathering and processing
  - Sentiment analysis

Tweet: *There is a conference on data science.*

Tweet: *I'm excited to attend the data science conference.*

There is a conference on data science.	Neutral
I'm excited to attend the conference.	Positive

Python

Python

- **Introduction**

- Python is the most widely used programming language for data science.
- Released in 1991 by Guido and Rossum
- Wors on different platforms (Windows, Mac, Linux, etc)
- Could be used both for functional or object-oriented programming
- Python has a default terminal to write your code. However, it is possible to write your code in an integrated development environment (IDE) such as Pycharm, Spyder, Eclipse, etc.
- You can also use the command line to write your python code.

Python

- **Basics**

- Python keywords and variables are case-sensitive. Print is different from print.
- Python uses indentation for distinguishing a block of codes.

Example

```
For i in range(5):
  print(i)
```

- For assigning a value to a variable we do not need to declare them. We can simply assign the value using an equal sign

Example

```
x=2
First='Hey guys'
```

- Comments in python start with a #.

Python

- **Variables**

- Variable name must start with a letter or underscore character
- Variable name cannot start with a number
- Variable names are case sensitive
- The operators +, -, *, / are used for basic mathematical operations add, subtraction, multiplication, and division.

Example

```
_my_name='Amin'
_My_name='amin'
x=2
y=5
z=x+y
z=_my_name+_My_name
```

NOTE: We can use the print() function to output variables.

```
Print(_my_name)
```

Python

- **Variables**

- Variables that are created outside of a function (as in all of the examples above) are known as global variables.

```
x = "awesome"
def myfunc():
  print("Python is " + x)
myfunc()
```

- If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function

```
x = "awesome"
def myfunc():
  x = "fantastic"
  print("Python is " + x)
myfunc()
print("Python is " + x)
```

Python

- **Variables**

- Normally, when you create a variable inside a function, that variable is local, and can only be used inside that function. To create a global variable inside a function, you can use the global keyword

```
def myfunc():
  global x
  x = "fantastic"
myfunc()
print("Python is " + x)
```

- Also, use the global keyword if you want to change a global variable inside a function.

```
x = "awesome"
def myfunc():
  global x
  x = "fantastic"
myfunc()
print("Python is " + x)
```

Python

- **Data types**

- Variables can store data of different types, and different types can do different things. Python has some data types that are built-in by default, the most important of which are as follows:

Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Boolean Type:	bool
None Type:	NoneType

NOTE: You can get the data type of any object by using the type() function:

```
x = 5
print(type(x))
```

Python

- **Data types**

- In Python, the data type is set when you assign a value to a variable
- If you want to specify the data type, you can use the following constructor functions

x = **str**("Hello World")	str
x = **int**(20)	int
x = **float**(20.5)	float
x = **list**(("apple", "banana", "cherry"))	list
x = **tuple**(("apple", "banana", "cherry"))	tuple
x = **dict**(name="John", age=36)	dict
x = **bool**(5)	bool

- These functions could be used to convert a data type to another data type

x=int(20.7)

Python

- **Numbers**

- There are three numeric types in Python: int, float, and complex

```
x=158
y=152.879
print(type(x))
print(type(y))
```

- We can use the constructor functions in the previous slide to convert some data types to other data types.

Python

- **Strings**

- Strings in python are surrounded by either single quotation marks, or double quotation marks. 'hello' is the same as "hello"

```
print("Hello")
print('Hello’)
-----------
a = "Hello"
print(a)
-----------
```

- You can assign a multiline string to a variable by using three quotes:

```
a = """“Today is Wednesday,
Yes, it is.,
Let’s learn Python."""
print(a)
```

Python

- **Strings**

- Python does not have a character data type, a single character is simply a string with a length of 1.
- Like many other popular programming languages, strings in Python are arrays of bytes representing Unicode characters. Square brackets can be used to access elements of the string.

```
a = "Hello, World!"
print(a[1])
```

- To get the length of a string, use the len() function.

```
a = "Hello, World!"
print(len(a))
```

- You can return a range of characters by using the slice syntax.

```
b = "Hello, World!"
print(b[2:5])
```

```
b = "Hello, World!"
print(b[2:])
```

Python

- **Strings**

- We can also use negative indexing. In normal indexing, the index starts from 0 and the first character. In negative indexing, the index starts from -1 and the last character.

```
b = "Hello, World!"
print(b[-5:-2])
```

- A few other examples (we will learn each of these examples later)

```
for x in "banana":
  print(x)
----
txt = "The best things in life are free!"
if "free" in txt:
  print("Yes, 'free' is present.")
----
txt = "The best things in life are free!"
if "expensive" not in txt:
  print("No, 'expensive' is NOT present.")
```

Python

- **Strings**

- The strip() method removes any whitespace from the beginning or the end.

```
a = " Hello, World! "
print(a.strip()) # returns "Hello, World!"
```

- To concatenate, or combine, two strings you can use the + operator.

```
a = "Hello"
b = "World"
c = a + b
print(c)
-------
a = "Hello"
b = "World"
c = a + " " + b
print(c)
```

Python

- **Strings**

- We cannot combine strings and numbers using +.

```
age = 36
txt = "My name is John, I am " + age
print(txt)
```

- We can combine strings and numbers by using the format() method

```
age = 36
txt = "My name is John, and I am {}"
print(txt.format(age))
```

Python

- **Strings**

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want {} pieces of item {} for {} dollars."
print(myorder.format(quantity, itemno, price))
```

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want to pay {2} dollars for {0} pieces of item {1}."
print(myorder.format(quantity, itemno, price))
```

Python

- **Operators**

- Arithmetic operators

+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
**	Exponentiation	x ** y.

- Comparison operators

==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Python

- **Operators**

- Logical operators

and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

- Other operators

is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Python

- **Lists**

- Lists are used to store multiple items in a single variable. The items could be of different data type.
- Lists are created using square brackets.

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
---
thislist = [1, "banana", "cherry"]
print(thislist)
---
```

- List items are ordered, changeable, and allow duplicate values. List items are indexed, the first item has index [0], the second item has index [1] etc. Lists can have items with the same value

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

Python

- **Lists**

- To determine how many items a list has, use the len() function

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

- A list can contain different data types

```
list1 = ["abc", 34, True, 40, "male"]
```

- It is also possible to use the list() constructor to create a new list

```
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

Python

- **Lists**

NOTE: There are four collection data types in the Python programming language:

List is a collection which is ordered and changeable. Allows duplicate members.
Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
Set is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
Dictionary is a collection which is ordered** and changeable. No duplicate members.

- List items are indexed and you can access them by referring to the index number

```
thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

NOTE: The first item has index 0.

- Negative indexing starts from the end. -1 refers to the last item, -2 refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]
print(thislist[-1])
```

Python

- **Lists**

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])
```

NOTE: In the above example, the search will start at index 2 (included) and end at index 5 (not included).

- To change the value of a specific item, refer to the index number

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist[1:3] = ["blackcurrant", "watermelon"]
print(thislist)
```

Python

- **Lists**

- The insert() method inserts an item at the specified index

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(2, "watermelon")
print(thislist)
```

- To add an item to the end of the list, use the append() method

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

- To append elements from another list to the current list, use the extend() method.

```
thislist = ["apple", "banana", "cherry"]
tropical = ["mango", "pineapple", "papaya"]
thislist.extend(tropical)
print(thislist)
```

Python

- **Lists**

- The remove() method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

- The pop() method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```

NOTE: If you do not specify the index, the pop() method removes the last item.

- List objects have a sort() method that will sort the list

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist)
```

Python

- **Lists**

```
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

- Sorting descending

```
thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse = True)
print(thislist)
```

- The reverse() method reverses the current sorting order of the elements

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```

Python

- **Lists**

NOTE: You cannot copy a list simply by typing list2 = list1, because: list2 will only be a reference to list1, and changes made in list1 will automatically also be made in list2.

- There are ways to make a copy, one way is to use the built-in List method copy()

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)
```

Python

- **Tuple**

- A tuple is a collection which is ordered and unchangeable.

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

- Tuples have the same properties as lists but tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
----
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:5])
----
```

Python

- **Tuple**

- To join two or more tuples you can use the + operator

```
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)

tuple3 = tuple1 + tuple2
print(tuple3)
```

NOTE: We can create a tuple with one elements as follows:

```
a=(1,)
```

NOTE: a=(1) will not create a tuple.

Python

- **Dictionaries**

- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates
- As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.
- Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Python

- **Dictionaries**

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])
```

- The keys() method will return a list of all the keys in the dictionary.

```
x = thisdict.keys()
```

- The values() method will return a list of all the values in the dictionary.

```
x = thisdict.values()
```

Python

- **Dictionaries**

- To determine if a specified key is present in a dictionary use the in keyword:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
if "model" in thisdict:
  print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

- You can change the value of a specific item by referring to its key name

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict["year"] = 2018
```

Python

- **Dictionaries**

- Adding an item to the dictionary is done by using a new index key and assigning a value to it

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict["color"] = "red"
print(thisdict)
```

- The pop() method removes the item with the specified key name

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.pop("model")
print(thisdict)
```

Python

- **IF**

- An "if statement" is written by using the if keyword. The elif keyword is pythons way of saying "if the previous conditions were not true, then try this condition". The else keyword catches anything which isn't caught by the preceding conditions.

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

Python

- **IF**

```
a = 200
b = 33
c = 500
if a > b and c > a:
  print("Both conditions are True")
```

- if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

```
a = 33
b = 200

if b > a:
  pass
```

Python

- **While**

- With the while loop we can execute a set of statements as long as a condition is true.

```
i = 1
while i < 6:
  print(i)
  i += 1
```

- With the break statement we can stop the loop even if the while condition is true

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

Python

- **For**

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

```
for x in "banana":
  print(x)
```

- With the break statement we can stop the loop before it has looped through all the items

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```

Python

- **For**

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

- To loop through a set of code a specified number of times, we can use the range() function.
- The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6)

```
for x in range(2, 6):
  print(x)
```

- The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, 3):

```
for x in range(2, 30, 3):
  print(x)
```

Python

- **List comprehension**

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]

newlist = [x for x in fruits if "a" in x]

print(newlist)
```

```
newlist = [x for x in fruits]

print(newlist)
```

Python

- **Function**

- A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.
- In Python a function is defined using the def keyword

```
def my_function():
  print("Hello from a function")
```

- To call a function, use the function name followed by parenthesis:

```
def my_function():
  print("Hello from a function")

my_function()
```

Python

- **Function**

- If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition. This way the function will receive a tuple of arguments, and can access the items accordingly

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

- Default parameter

```
def my_function(country = "Norway"):
  print("I am from " + country)

my_function()
my_function("Brazil")
```

Python

- **Function**

- You can send any data types of argument to a function (string, number, list, dictionary etc.)

```
def my_function(food):
  for x in food:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

- To let a function return a value, use the return statement

```
def my_function(x):
  return 5 * x

print(my_function(3))
print(my_function(5))
```

Python

- **Module**

- A module in python is a code library containing a set of functions you want to include in your application.
- There are different modules for different applications. To create a module just save the code you want in a file with the file extension .py
- We can use modules by using the import statement

Assume we save the following code in a file named mymodule.py.

```
def greeting(name):
  print("Hello, " + name)
```

We can call the greeting function as follows:

```
import mymodule
mymodule.greeting("Jonathan")
```

Python

- **Module**

- We can access variables inside a module as follows:

Save the following code in a file named mymodule.py

```
person1 = {
  "name": "John",
  "age": 36,
  "country": "Norway"
}
```

```
import mymodule
a = mymodule.person1["age"]
print(a)
```

```
import mymodule as mx
a = mx.person1["age"]
print(a)
```

Python Pandas

- **Introduction**
- Pandas is a Python library used for working with data sets.
- It has functions for analyzing, cleaning, exploring, and manipulating data.
- Pandas allows us to analyze big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.
- The source code for Pandas is located at this github repository https://github.com/pandas-dev/pandas

Python Pandas

- **Introduction**

- If you have Python and PIP already installed on a system, then installation of Pandas is very easy.
- You can install Pandas on your system by using pip as follows:

```
pip install pandas
```

- You can import Pandas as follows:

```
import pandas
```

- Pandas is usually imported under the pd alias.

```
import pandas as pd
```

- You can check Pandas version as follows:

```
import pandas as pd
print(pd.__version__)
```

Python Pandas

- **DataFrames**

- A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.
- You can use the DataFrame() method to create a Pandas DataFrame from a dictionary, numpy array, or a list.

```
import pandas as pd
data = {"calories": [420, 380, 390], "duration": [50, 40, 45]}
#load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
---
data = [[420, 380, 390],[50, 40, 45]]
df = pd.DataFrame(data)
print(df)
```

Python Pandas

- **DataFrames indexing**

- You can use loc() or iloc() to return values, rows, or columns inside a data frame. Using loc() you need to specify the column name, but using iloc() you only need to specify the column index.

```
import pandas as pd
data = {"calories": [420, 380, 390], "duration": [50, 40, 45]}
#load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
print("\n")
print(df.loc[0,'calories'])
print("\n")
print(df.loc[0,:])
print("\n")
print(df.loc[:,'calories'])
```

Python Pandas

- **DataFrames indexing**

- You can use loc() or iloc() to return values, rows, or columns inside a data frame. Using loc() you need to specify the column name, but using iloc() you only need to specify the column index.

```
import pandas as pd
data = {"calories": [420, 380, 390], "duration": [50, 40, 45]}
#load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
print("\n")
print(df.iloc[0,0])
print("\n")
print(df.iloc[0,:])
print("\n")
print(df.iloc[:,0])
```

Python Pandas

- **DataFrames indexing**

- You can get the indices and columns names using index and column attributes as follows

```
import pandas as pd
data = {"calories": [420, 380, 390], "duration": [50, 40, 45]}
#load data into a DataFrame object:
df = pd.DataFrame(data)

index_names=list(df.index)
column_names=list(df.columns)

print(index_names,end='\n\n')
print(column_names)
```

Python Pandas

- **DataFrames indexing**

- With the index and columns argument, you can name your own indexes and columns.

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```

Python Pandas

- **DataFrames indexing**

- With the index argument, you can name your own indexes and columns.

```
import pandas as pd

data =[[420, 380, 390],[50, 40, 45]]

df = pd.DataFrame(data, columns = ["c1", "c2", "c3"])

print(df)
```

Python Pandas

- **Reading CSV and JSON**

- You can import CSV and JSON files into a Pandas DataFrame using read_csv() and read_json() as follows:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df)
```

```
import pandas as pd
df = pd.read_json('data.json')
print(df)
```